TRIALGO — Workflow Complet des Cartes et Relations

> Ce document détaille le coeur du système TRIALGO : comment les cartes sont structurées, reliées entre elles, et utilisées pour générer des questions de jeu.

Table des matières

1. Les 3 types de cartes
2. La formule fondamentale E (+) C = R
3. Les 2 mécanismes de relation
4. Le système de distances (D1, D2, D3)
5. Exemple complet : la chaîne du Lion
6. Le stockage en base de données
7. La génération d'une question
8. La sélection des distracteurs
9. La validation d'une réponse
10. Les 3 configurations de question (A, B, C)
11. Scénarios de jeu complets
12. Cas limites et règles spéciales

---

1. Les 3 types de cartes

Chaque carte du jeu est une **image dessinée par un artiste**. Il n'y a pas de texte, pas de code, que des images. Le joueur raisonne visuellement.

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│   ÉMETTRICE (E)              CÂBLE (C)           RÉCEPTRICE (R)      │
│   Image de base              Image de             Image résultat     │
│                              transformation                          │
│   ┌──────────┐               ┌──────────┐        ┌──────────┐       │
│   │          │               │          │        │          │       │
│   │    🦁    │               │    ↔     │        │   🦁↔    │       │
│   │  Lion    │               │  Miroir  │        │Lion Mir. │       │
│   │          │               │          │        │          │       │
│   └──────────┘               └──────────┘        └──────────┘       │
│                                                                      │
│   Exemples :                 Exemples :          Exemples :          │
│   - Lion                     - Miroir horizontal - Lion miroir       │
│   - Aigle                    - Rotation 90°      - Aigle rouge       │
│   - Requin                   - Teinte rouge      - Requin frag.      │
│   - Éléphant                 - Fragmentation     - Éléph. rot.       │
│                                                                      │
│   Stockage :                 Stockage :          Stockage :          │
```

```
│   emettrices/{theme}/        cables/{catégorie}/ receptrices/   │
│   lion_base.webp             miroir_h.webp       {theme}/d{n}/  │
│                                                  lion_miroir.wbp│
└──────────────────────────────────────────────────────────────────┘
```

Propriétés spécifiques par type

```
┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐
│ Propriété        │ Émettrice        │ Câble            │ Réceptrice       │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ card_type        │ "emettrice"      │ "cable"          │ "receptrice"     │
│ cableCategory    │ null             │ "geometrique"    │ null             │
│                  │                  │ "couleur"        │                  │
│                  │                  │ "dimension"      │                  │
│                  │                  │ "complexe"       │                  │
│ parentEmettri.   │ null             │ null             │ UUID de l'E ou R │
│ parentCableId    │ null             │ null             │ UUID du C utilisé│
│ rootEmettri.     │ null             │ null             │ UUID de l'E rac. │
│ themeTags        │ [animal, lion,   │ [miroir,         │ [animal, lion,   │
│                  │  savane, félin]  │  symétrie]       │  savane, miroir] │
└──────────────────┴──────────────────┴──────────────────┴──────────────────┘
```

Règle clé : seules les **Réceptrices** ont des parents (`parentEmettriceId`, `parentCableId`). Les Émettrices et les Câbles sont des cartes **indépendantes**.

---

2. La formule fondamentale E (+) C = R

Le coeur de TRIALGO tient en une formule :

```
Émettrice  (+)  Câble  =  Réceptrice
   E        +     C    =      R

Image      +  Transfor-  =  Image
de base       mation        résultat
```

Chaque combinaison valide `(E, C, R)` est appelée un **trio**. C'est l'unité fondamentale du jeu.

Exemples de trios

```
Trio 1 :  Lion     + Miroir H.    = Lion Miroir
Trio 2 :  Aigle    + Rotation 90° = Aigle Rotation
Trio 3 :  Requin   + Teinte Rouge = Requin Rouge
Trio 4 :  Éléphant + Fragment. 3  = Éléphant Fragmenté
Trio 5 :  Renard   + Niv. de gris = Renard Gris
```

Important : la combinaison `Lion + Rotation` ne donne PAS `Lion Miroir`. Chaque trio est **unique et prédéfini** par l'artiste. On ne peut pas combiner n'importe quoi. La table `card_trios` est la **source de vérité**.

---

3. Les 2 mécanismes de relation

Le système utilise **deux mécanismes complémentaires** pour relier les cartes :

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║   MÉCANISME 1 : card_trios                                    ║
║   ────────────────────────                                    ║
║   RÔLE : Définir les combinaisons VALIDES pour le JEU         ║
║                                                               ║
║   Contient : emettrice_id + cable_id + receptrice_id          ║
║   Question : "Est-ce que E + C = R ?"                         ║
║   Utilisé par : Génération de questions, validation de réponses ║
║                                                               ║
║   C'est la TABLE DE JEU. Sans elle, pas de partie.            ║
║                                                               ║
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║   MÉCANISME 2 : auto-références dans cards                    ║
║   ────────────────────────────────────────                    ║
║   RÔLE : Tracer la FILIATION des images pour la pédagogie     ║
║                                                               ║
║   Contient : parent_emettrice_id, parent_cable_id, root_e_id  ║
║   Question : "D'où vient cette image ?"                       ║
║   Utilisé par : Galerie, affichage des chaînes, requêtes rapides ║
║                                                               ║
║   C'est l'ARBRE GÉNÉALOGIQUE des images.                      ║
║                                                               ║
╔═══════════════════════════════════════════════════════════════╗
```

Pourquoi deux mécanismes ?

```
Scénario : le joueur est au niveau 8 (Distance 2).
Le jeu lui pose une question sur le trio D2 du Lion.

POUR JOUER (mécanisme 1 — card_trios) :
  → Le jeu a besoin de savoir que R1 + C2 = R2
  → Il lit card_trios : { e: R1, c: C2, r: R2 }
  → Il génère la question et les distracteurs
  → Le joueur répond, on vérifie dans card_trios

POUR EXPLIQUER (mécanisme 2 — auto-références) :
  → Le joueur veut comprendre d'où vient R2
  → On lit R2.parentEmettriceId → R1 (Lion Miroir)
  → On lit R2.parentCableId → C2 (Teinte Rouge)
```

```
    → On lit R2.rootEmettriceId → E1 (Lion, la racine)
    → On affiche : "Lion → Miroir → Lion Miroir → Rouge → Lion Miroir
Rouge"
```

---

4. Le système de distances (D1, D2, D3)

Les distances créent des **chaînes de transformations** de plus en plus longues.

Distance 1 — Trio simple (3 images)

```
┌──────┐     ┌──────┐     ┌──────┐
│  E1  │ (+) │  C1  │ (=) │  R1  │
│Lion  │     │Miroi│     │Lion  │
│base  │     │r H.  │     │Mir.  │
└──────┘     └──────┘     └──────┘

3 images, 1 trio
Niveaux 1 à 6
```

Distance 2 — Quintette (5 images, 2 trios)

```
┌──────┐     ┌──────┐     ┌──────┐     ┌──────┐     ┌──────┐
│  E1  │ (+) │  C1  │ (=) │  R1  │ (+) │  C2  │ (=) │  R2  │
│Lion  │     │Miroi│     │Lion  │     │Rouge│     │Lion  │
│base  │     │r H.  │     │Mir.  │     │      │     │M.Rg  │
└──────┘     └──────┘     └──────┘     └──────┘     └──────┘
                             │
                             │ R1 DEVIENT l'Émettrice
                             │ du trio suivant
                             ▼
                   Trio D1 : E1 + C1 = R1
                   Trio D2 : R1 + C2 = R2

5 images uniques, 2 trios liés
Niveaux 7 à 14
```

Distance 3 — Septette (7 images, 3 trios)

```
┌─────┐   ┌─────┐   ┌─────┐   ┌─────┐   ┌─────┐   ┌─────┐   ┌─────┐
│ E1  │(+)│ C1  │(=)│ R1  │(+)│ C2  │(=)│ R2  │(+)│ C3  │(=)│ R3  │
│Lion│   │Mir.│   │Lion│   │Rou.│   │Lion│   │Fra.│   │Lion│
│    │   │    │   │Mir.│   │    │   │M.Rg│   │    │   │Ful.│
└─────┘   └─────┘   └─────┘   └─────┘   └─────┘   └─────┘   └─────┘
                      │                   │
                      ▼                   ▼
```

```
                Trio D1: E1+C1=R1   Trio D2: R1+C2=R2   Trio D3: R2+C3=R3

  7 images uniques, 3 trios liés
  Niveaux 15+
```

Règle fondamentale des distances

```
  À chaque distance, la Réceptrice du trio précédent
  DEVIENT l'Émettrice du trio suivant :

  D1 : E1 + C1 = R1
  D2 : R1 + C2 = R2     ← R1 joue le rôle d'E
  D3 : R2 + C3 = R3     ← R2 joue le rôle d'E

  Dans card_trios :
    Trio D2.emettrice_id = uuid-R1   (pas un type "emettrice", mais une
Réceptrice !)
    Trio D3.emettrice_id = uuid-R2
```

Attention : en D2 et D3, l'`emettrice_id` du trio pointe vers une carte dont le `card_type` est `"receptrice"`. C'est **normal**. Le trio définit les **rôles** (E, C, R), pas les types SQL des cartes.

---

5. Exemple complet : la chaîne du Lion

Voici toutes les données telles qu'elles apparaissent en base pour une chaîne complète.

5.1. Les 7 cartes dans la table `cards`

```
┌──────────┬──────────┬────┬──────────────────────────────┬────────
──┬──────────┬──────────┐
│ id       │ card_type │ dist │ image_path                   │
parent_e │ parent_c │ root_e   │
├──────────┼──────────┼────┼──────────────────────────────┼────────
──┼──────────┼──────────┤
│ E1-uuid  │ emettrice │  1   │ emettrices/savane/lion_base.webp│  null
│  null    │  null    │
│ C1-uuid  │ cable     │  1   │ cables/geometrique/miroir_h.webp│  null
│  null    │  null    │
│ R1-uuid  │ receptrice │  1   │ receptrices/savane/d1/         │ E1-
uuid │ C1-uuid │  null    │
│          │          │      │ lion_miroir_h.webp             │
│          │          │
│ C2-uuid  │ cable     │  2   │ cables/couleur/teinte_rouge.webp│  null
│  null    │  null    │
│ R2-uuid  │ receptrice │  2   │ receptrices/savane/d2/         │ R1-
uuid │ C2-uuid │ E1-uuid │
│          │          │      │ lion_miroir_h_rouge.webp       │
│          │          │
```

```
  | C3-uuid   | cable      |  3   | cables/complexe/fragment_3.webp |   null
  |   null    |   null    |
  | R3-uuid   | receptrice |  3   | receptrices/savane/d3/          |  R2-
uuid  |  C3-uuid  |  E1-uuid  |
  |           |            |      | lion_miroir_h_rouge_frag.webp   |
  |           |           |
  └───────────┴────────────┴──────┴─────────────────────────────────┴────────
  ──┴────────────┴────────────┘
```

5.2. Lecture des auto-références

```
  QUESTION : "D'où vient R3 (Lion Miroir Rouge Fragmenté) ?"

  R3.parentEmettriceId = R2-uuid
    → R2 = Lion Miroir Rouge       "R3 a été créée à partir de R2"

  R3.parentCableId = C3-uuid
    → C3 = Fragmentation 3         "en appliquant la fragmentation"

  R3.rootEmettriceId = E1-uuid
    → E1 = Lion de base            "la racine de toute la chaîne est le
Lion"


  QUESTION : "D'où vient R2 (Lion Miroir Rouge) ?"

  R2.parentEmettriceId = R1-uuid
    → R1 = Lion Miroir              "R2 a été créée à partir de R1"

  R2.parentCableId = C2-uuid
    → C2 = Teinte Rouge             "en appliquant la teinte rouge"

  R2.rootEmettriceId = E1-uuid
    → E1 = Lion de base             "même racine"


  QUESTION : "D'où vient R1 (Lion Miroir) ?"

  R1.parentEmettriceId = E1-uuid
    → E1 = Lion de base             "R1 a été créée à partir de E1"

  R1.parentCableId = C1-uuid
    → C1 = Miroir Horizontal        "en appliquant le miroir"

  R1.rootEmettriceId = null
    → null car le parent direct (E1) EST déjà la racine.
      Pas besoin de raccourci.
```

5.3. Les 3 trios dans `card_trios`

```
┌──────────┬──────────────┬──────────┬────────────────┬──────┬────────────┬
──────┐
│ id       │ emettrice_id │ cable_id │ receptrice_id  │ dist │ parent_trio│
diff │
├──────────┼──────────────┼──────────┼────────────────┼──────┼────────────┼
──────┤
│ T1-uuid  │   E1-uuid    │ C1-uuid  │   R1-uuid      │  1   │    null    │
0.3  │
│ T2-uuid  │   R1-uuid    │ C2-uuid  │   R2-uuid      │  2   │  T1-uuid   │
0.5  │
│ T3-uuid  │   R2-uuid    │ C3-uuid  │   R3-uuid      │  3   │  T2-uuid   │
0.8  │
└──────────┴──────────────┴──────────┴────────────────┴──────┴────────────┴
──────┘

  Lecture :
    T1 : E1 + C1 = R1                (trio racine, pas de parent)
    T2 : R1 + C2 = R2, fils de T1    (R1 joue le rôle de E)
    T3 : R2 + C3 = R3, fils de T2    (R2 joue le rôle de E)
```

5.4. Reconstitution de la chaîne complète

```
  Pour afficher la chaîne dans la galerie ou le tutoriel :

  Départ : T3-uuid (le trio D3 du lion)

  Étape 1 : T3.parentTrioId = T2-uuid → charger T2
  Étape 2 : T2.parentTrioId = T1-uuid → charger T1
  Étape 3 : T1.parentTrioId = null    → fin de la chaîne

  Résultat :
    T1 (D1) → T2 (D2) → T3 (D3)

  Ou avec rootEmettriceId (raccourci, 1 seule requête) :
    R3.rootEmettriceId = E1-uuid → on sait immédiatement
    que le Lion de base est l'ancêtre de tout.
```

---

6. Le stockage en base de données

6.1. Schéma SQL des relations

```
  ┌──────────────────────────────────────────────────────┐
  │                      cards                           │
  │                                                      │
  │  id ──────────────────────────────────────────┐      │
  │  card_type                                    │      │
  │  distance_level                               │      │
```

```
    │  image_path                                    │    │
    │  parent_emettrice_id ──FK──┐                   │    │
    │  parent_cable_id ──────FK──┤  auto-           │    │
    │  root_emettrice_id ────FK──┘  références      │    │
    │  cable_category                vers cards.id  │    │
    │  theme_tags                                    │    │
    │  ...                                           │    │
    └────────────────────────────────────────────────┘    │
                                │                         │
                                │                         │
                                └─────────────────────────┘

                                          │
              ┌───────────────────────────┼──────────────────────────┐
              │                           │                          │
              ▼                           ▼                          ▼

    ┌─────────────────┐       ┌─────────────────┐       ┌─────────────────┐
    │  card_trios     │       │ user_unlocked   │       │ broken_image    │
    │                 │       │ _cards          │       │ _reports        │
    │ emettrice_id──FK        │ card_id───FK    │       │ card_id───FK    │
    │ cable_id──────FK        │ user_id───FK    │       │ reported_by─FK  │
    │ receptrice_id─FK        └─────────────────┘       └─────────────────┘
    │ parent_trio_id─FK
    │   (auto-ref)
    └─────────────────┘
```

6.2. Contraintes SQL importantes

```sql
-- Un trio est unique : pas de doublons E+C+R
UNIQUE (emettrice_id, cable_id, receptrice_id)

-- Toutes les FK pointent vers des cartes existantes
emettrice_id   UUID NOT NULL REFERENCES cards(id)
cable_id       UUID NOT NULL REFERENCES cards(id)
receptrice_id  UUID NOT NULL REFERENCES cards(id)

-- Les auto-références sont optionnelles (null pour E et C)
parent_emettrice_id UUID REFERENCES cards(id)   -- nullable
parent_cable_id     UUID REFERENCES cards(id)   -- nullable
root_emettrice_id   UUID REFERENCES cards(id)   -- nullable

-- Distance entre 1 et 3
CHECK (distance_level BETWEEN 1 AND 3)
```

6.3. Index pour la performance

```sql
-- Recherche rapide par type (pour les distracteurs)
CREATE INDEX idx_cards_type ON cards(card_type);

-- Recherche rapide par distance (pour la génération)
CREATE INDEX idx_cards_distance ON cards(distance_level);
```

```sql
-- Filtrer les cartes actives (toutes les requêtes de jeu)
CREATE INDEX idx_cards_active ON cards(is_active);

-- Trouver les trios par distance (pour la génération)
CREATE INDEX idx_trios_distance ON card_trios(distance_level);
```

---

7. La génération d'une question

Voici le processus complet, du clic "Jouer" jusqu'à l'affichage.

```
Le joueur appuie sur "Jouer" — Niveau 7
═══════════════════════════════════════

┌──────────────────────────────────────────────────────────────┐
│  ÉTAPE 1 — Déterminer les règles du niveau                   │
│                                                              │
│  GameConstants.getLevelConfig(7) :                           │
│                                                              │
│    distance       = 2         → quintettes (D2)              │
│    configs        = ['A','B'] → E+C=? ou E+?=R               │
│    questions      = 10        → 10 questions dans le niveau  │
│    threshold      = 7         → 7/10 pour réussir            │
│    turnTimeSeconds = 40       → 40 secondes par question     │
│    basePoints     = 20        → 20 pts par bonne réponse     │
│    livesPerWrong  = 2         → 1 vie perdue pour 2 erreurs  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
                                 │
                                 │
                                 ▼
┌──────────────────────────────────────────────────────────────┐
│  ÉTAPE 2 — Tirer un trio aléatoire                           │
│                                                              │
│  Requête Supabase :                                          │
│                                                              │
│    SELECT * FROM card_trios                                  │
│    WHERE distance_level = 2                                  │
│      AND id NOT IN ('T1-uuid', 'T5-uuid')                    │
│    LIMIT 10                                        ↑         │
│                                     trios déjà posés         │
│                                     dans cette session       │
│                                                              │
│  Résultat : 10 trios D2 disponibles                          │
│  Choix aléatoire : T2-uuid                                   │
│    → { e: R1-uuid, c: C2-uuid, r: R2-uuid }                  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
                                 │
                                 │
                                 ▼
┌──────────────────────────────────────────────────────────────┐
```

```
    │  ÉTAPE 3 — Charger les 3 cartes EN PARALLÈLE                │
    │                                                             │
    │  Future.wait([                                              │
    │    getCardById(R1-uuid),   ← requête 1                      │
    │    getCardById(C2-uuid),   ← requête 2  (en même temps)     │
    │    getCardById(R2-uuid),   ← requête 3  (en même temps)     │
    │  ])                                                         │
    │                                                             │
    │  Résultat (en ~100ms au lieu de ~300ms) :                   │
    │    emettrice  = R1 (Lion Miroir — joue le rôle de E)        │
    │    cable      = C2 (Teinte Rouge)                           │
    │    receptrice = R2 (Lion Miroir Rouge)                      │
    │                                                             │
    └─────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
    ┌─────────────────────────────────────────────────────────────┐
    │  ÉTAPE 4 — Choisir la configuration                         │
    │                                                             │
    │  configs disponibles = ['A', 'B']                           │
    │  Tirage aléatoire → 'B'                                     │
    │                                                             │
    │  Config B signifie :                                        │
    │    visible  = [Émettrice (R1), Réceptrice (R2)]             │
    │    masquée  = Câble (C2)                                    │
    │    Question = "Quelle transformation relie ces images ?"    │
    │                                                             │
    └─────────────────────────────────────────────────────────────┘
                                  │
                                  │
                                  ▼
    ┌─────────────────────────────────────────────────────────────┐
    │  ÉTAPE 5 — Générer 9 distracteurs                           │
    │  (voir section 8 pour le détail)                            │
    │                                                             │
    │  La carte masquée est C2 (Câble, catégorie "couleur")       │
    │                                                             │
    │  Requête 1 — même catégorie :                               │
    │    SELECT FROM cards                                        │
    │    WHERE card_type = 'cable'                                │
    │      AND cable_category = 'couleur'                         │
    │      AND id != C2-uuid                                      │
    │    LIMIT 4                                                  │
    │    → [Teinte Bleu, Niv.Gris, Sépia, Inversion]              │
    │                                                             │
    │  Requête 2 — autres catégories :                            │
    │    SELECT FROM cards                                        │
    │    WHERE card_type = 'cable'                                │
    │      AND cable_category != 'couleur'                        │
    │      AND id != C2-uuid                                      │
    │    LIMIT 5                                                  │
    │    → [Miroir V, Rot.90, Fragment 2, Zoom, Ombre]            │
    │                                                             │
    │  Fusion + mélange : 9 distracteurs                          │
    │                                                             │
```

```
└──────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌──────────────────────────────────────────────────────────┐
│  ÉTAPE 6 — Assembler les 10 choix                        │
│                                                          │
│  choices = [C2, TeinteBleu, Niv.Gris, Sépia, Inversion,  │
│             MiroirV, Rot90, Fragment2, Zoom, Ombre]      │
│                                                          │
│  choices.shuffle() → mélange aléatoire :                 │
│                                                          │
│  [Rot90, TeinteBleu, Ombre, C2, Fragment2,               │
│   Sépia, Zoom, MiroirV, Niv.Gris, Inversion]             │
│         position 3 ──────┘                               │
│         (la bonne réponse est à un endroit aléatoire)    │
│                                                          │
└──────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌──────────────────────────────────────────────────────────┐
│  ÉTAPE 7 — Retourner GameQuestionEntity                  │
│                                                          │
│  GameQuestionEntity(                                     │
│    visibleCards     : [R1 (Lion Miroir), R2 (Lion M. Rge)], │
│    maskedCard       : C2 (Teinte Rouge),                 │
│    choices          : [10 câbles mélangés],              │
│    config           : 'B',                               │
│    correctCardId    : C2-uuid,                           │
│    trioId           : T2-uuid,                           │
│    timeLimitSeconds : 40,                                │
│  )                                                       │
└──────────────────────────────────────────────────────────┘
                                │
                                │
                                ▼
┌──────────────────────────────────────────────────────────┐
│  AFFICHAGE À L'ÉCRAN                                     │
│                                                          │
│  ┌──────────┐        ┌──────────┐        ┌──────────┐   │
│  │ R1       │   (+)  │   ???    │   (=)  │ R2       │   │
│  │Lion Mir. │        │          │        │Lion M.Rge│   │
│  └──────────┘        └──────────┘        └──────────┘   │
│                                                          │
│  "Quelle transformation relie ces deux images ?"         │
│                                                          │
│  ┌────┐┌────┐┌────┐┌────┐┌────┐┌────┐┌────┐┌────┐┌────┐┌────┐
│  │Rot.││T.Bl││Ombr││T.Rg││Frg.││Sépi││Zoom││Mi.V││N.Gr││Inv.│
│  │    ││    ││    ││ ✓  ││    ││    ││    ││    ││    ││    │
│  └────┘└────┘└────┘└────┘└────┘└────┘└────┘└────┘└────┘└────┘
│                        ↑                                 │
│                 bonne réponse                            │
│           (position inconnue du joueur)                  │
└──────────────────────────────────────────────────────────┘
```

8. La sélection des distracteurs

Les distracteurs sont les 9 images **incorrectes** parmi les 10 choix. Leur sélection est **critique** pour la qualité du jeu.

8.1. Règle universelle

```
Les 9 distracteurs sont TOUJOURS du MÊME TYPE que la carte masquée :

┌──────────────┬──────────────────────┬──────────────────────────────┐
│ Config       │ Carte masquée        │ 9 distracteurs               │
├──────────────┼──────────────────────┼──────────────────────────────┤
│ A (facile)   │ Réceptrice           │ 9 autres Réceptrices         │
│ B (moyen)    │ Câble                │ 9 autres Câbles              │
│ C (dur)      │ Émettrice            │ 9 autres Émettrices          │
└──────────────┴──────────────────────┴──────────────────────────────┘

Pourquoi le même type ?
  → Si la masquée est un Câble et les distracteurs sont des Émettrices,
    le joueur n'a qu'à chercher "l'image qui ressemble à un câble".
    Trop facile. Il faut que les 10 choix soient visuellement homogènes.
```

8.2. Stratégie pour les Réceptrices (config A)

```
Masquée : R2 (Lion Miroir Rouge, distance 2)

SELECT * FROM cards
WHERE card_type = 'receptrice'        ← même type
  AND distance_level = 2              ← même distance (visuellement
proches)
  AND is_active = true                ← seulement les cartes actives
  AND id != R2-uuid                   ← exclure la bonne réponse
LIMIT 9

Résultat possible :
  Aigle Rotation Rouge, Requin Gris, Éléphant Miroir Rouge,
  Renard Fragment, Lion Rotation (D2), Panda Gris,
  Girafe Miroir Rouge, Baleine Rotation, Renard Miroir Rouge

Pourquoi même distance ?
  → Les réceptrices D2 sont toutes des images avec 2 transformations.
    Elles se ressemblent davantage entre elles qu'avec des D1.
    Le joueur doit vraiment ANALYSER pour trouver la bonne.
```

8.3. Stratégie pour les Câbles (config B) — stratégie en 2 temps

```
Masquée : C2 (Teinte Rouge, catégorie "couleur")

┌──────────────────────────────────────────────────────────────┐
│  REQUÊTE 1 : même catégorie (4 câbles "couleur")             │
│                                                              │
│  SELECT FROM cards                                           │
│  WHERE card_type = 'cable'                                   │
│    AND cable_category = 'couleur'                            │
│    AND id != C2-uuid                                         │
│  LIMIT 4                                                     │
│                                                              │
│  → Teinte Bleu, Niveaux de Gris, Sépia, Inversion            │
│                                                              │
│  Ces 4 câbles sont VISUELLEMENT PROCHES de Teinte Rouge.     │
│  Ce sont tous des transformations de couleur.                 │
│  Le joueur doit bien observer pour différencier.              │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│  REQUÊTE 2 : autres catégories (5 câbles différents)         │
│                                                              │
│  SELECT FROM cards                                           │
│  WHERE card_type = 'cable'                                   │
│    AND cable_category != 'couleur'                           │
│    AND id != C2-uuid                                         │
│  LIMIT 5                                                     │
│                                                              │
│  → Miroir V, Rotation 90, Fragment 2, Zoom x2, Ombre         │
│                                                              │
│  Ces 5 câbles sont d'autres catégories (géométrique,         │
│  dimension, complexe). Ils servent à compléter à 9.          │
└──────────────────────────────────────────────────────────────┘

FUSION :
  [T.Bleu, N.Gris, Sépia, Inversion, Miroir V,
   Rot.90, Frag.2, Zoom, Ombre]

SHUFFLE (mélange) :
  [Rot.90, T.Bleu, Ombre, Frag.2, Sépia,
   Zoom, Miroir V, N.Gris, Inversion]

Ratio : 4/9 proches visuellement + 5/9 différents
  → Le joueur ne peut pas éliminer par catégorie à coup sûr
  → Mais les plus proches augmentent la difficulté
```

8.4. Stratégie pour les Émettrices (config C)

```
Masquée : E1 (Lion, distance 1)

SELECT * FROM cards
```

```
    WHERE card_type = 'emettrice'
      AND distance_level = 1
      AND is_active = true
      AND id != E1-uuid
    LIMIT 9

    Résultat possible :
      Aigle, Requin, Éléphant, Renard, Panda,
      Girafe, Baleine, Loup, Ours

    Pas de sous-stratégie ici : toutes les Émettrices D1
    sont des images de base d'animaux. La difficulté vient
    du fait que le joueur voit C + R et doit deviner E.
```

---

9. La validation d'une réponse

9.1. Validation locale (aperçu côté client)

```
    Le joueur tape sur une image dans la ScrollView.

    ┌──────────────────────────────────────────────────────────────┐
    │                                                              │
    │  selectedCard.id == question.correctCardId ?                 │
    │                                                              │
    │  OUI → bonne réponse                                         │
    │    → animation verte sur la carte sélectionnée               │
    │    → score += basePoints * timeBonus * distanceMultiplier    │
    │    → streak += 1                                             │
    │                                                              │
    │  NON → mauvaise réponse                                      │
    │    → animation rouge sur la carte sélectionnée               │
    │    → animation verte sur la bonne réponse (révélation)       │
    │    → streak = 0                                              │
    │    → compteur erreurs += 1                                   │
    │    → si erreurs atteint livesPerWrong → lives -= 1           │
    │                                                              │
    │  TIMEOUT → pas de réponse                                    │
    │    → même traitement que mauvaise réponse                    │
    │    → malus supplémentaire : -5 points                        │
    │                                                              │
    └──────────────────────────────────────────────────────────────┘
```

9.2. Validation serveur (Edge Function — à déployer)

```
    En production, la vraie validation se fait CÔTÉ SERVEUR
    pour empêcher la triche :

    Client → POST /validate-answer
```

```
      {
        "trio_id": "T2-uuid",
        "selected_card_id": "C2-uuid",
        "config": "B",
        "elapsed_seconds": 12
      }

    Serveur → vérifie dans card_trios :
      SELECT EXISTS (
        SELECT 1 FROM card_trios
        WHERE id = 'T2-uuid'
          AND cable_id = 'C2-uuid'    ← config B = chercher le câble
      )

    Serveur → réponse :
      {
        "correct": true,
        "points_earned": 30,
        "bonus": "BONUS_TURBO"
      }
```

---

10. Les 3 configurations de question (A, B, C)

Config A — Trouver la Réceptrice (facile)

```
    Le joueur voit E et C, il doit trouver R.
    "Quel est le RÉSULTAT de cette transformation ?"

    ┌──────────┐          ┌──────────┐          ┌──────────┐
    │   🦁     │   (+)    │    ↔     │   (=)    │   ???    │
    │  Lion    │          │  Miroir  │          │          │
    │ (visible)│          │ (visible)│          │ (masqué) │
    └──────────┘          └──────────┘          └──────────┘

    Raisonnement du joueur :
      "Je vois un lion et un miroir horizontal.
       Le résultat doit être un lion retourné horizontalement.
       Parmi les 10 images, laquelle montre ça ?"

    Niveaux : 1 à 6 (introduction)
    Pourquoi c'est facile : le joueur a les 2 indices (source +
  transformation)
```

Config B — Trouver le Câble (moyen)

```
    Le joueur voit E et R, il doit trouver C.
    "Quelle TRANSFORMATION relie ces deux images ?"
```

```
┌──────────┐        ┌──────────┐        ┌──────────┐
│   🦁     │   (+)  │   ???    │   (=)  │   🦁↔    │
│  Lion    │        │          │        │Lion Mir. │
│ (visible)│        │ (masqué) │        │ (visible)│
└──────────┘        └──────────┘        └──────────┘

Raisonnement du joueur :
  "Je vois un lion normal et un lion retourné.
   Quelle transformation fait ça ?
   C'est forcément un miroir. Mais lequel ?
   Horizontal ? Vertical ? Parmi les 10 câbles..."

Niveaux : 4 à 18
Pourquoi c'est moyen : le joueur voit l'avant/après mais doit
  identifier visuellement le TYPE de transformation
```

Config C — Trouver l'Émettrice (difficile)

```
Le joueur voit C et R, il doit trouver E.
"Quelle est l'image DE DÉPART ?"

┌──────────┐        ┌──────────┐        ┌──────────┐
│   ???    │   (+)  │    ↔     │   (=)  │   🦁↔    │
│          │        │  Miroir  │        │Lion Mir. │
│ (masqué) │        │ (visible)│        │ (visible)│
└──────────┘        └──────────┘        └──────────┘

Raisonnement du joueur :
  "Je vois un miroir horizontal et un lion retourné.
   Si je 'déapplique' le miroir, quelle image j'obtiens ?
   Ce doit être un lion normal.
   Mais lequel parmi les 10 émettrices ?"

Niveaux : 8+
Pourquoi c'est difficile : le joueur doit raisonner "à l'envers",
  en déduisant l'image originale à partir du résultat et de la
transformation.
  C'est un raisonnement d'INVERSION mentale.
```

Tableau récapitulatif par niveau

```
┌─────────┬──────────┬─────────┬──────────────────────────┐
│ Niveaux │ Distance │ Configs │ Description              │
├─────────┼──────────┼─────────┼──────────────────────────┤
│  1 - 3  │   D1     │    A    │ Introduction : trouver R │
│  4 - 6  │   D1     │   A+B   │ Découverte config B      │
│  7 - 10 │   D2     │   A+B   │ Introduction quintettes  │
│ 11 - 14 │   D2     │    B    │ Câble exclusif           │
│ 15 - 18 │   D3     │   B+C   │ Introduction config C    │
```

```
    | 19 - 22 |   D3     |    C     | Expert : deviner E           |
    |   23+   |   D3     |  A+B+C   | Maître : toutes configs      |
    └─────────┴──────────┴──────────┴──────────────────────────────┘
```

---

11. Scénarios de jeu complets

Scénario 1 — Niveau 2, Config A, Bonne réponse

```
    Joueur : "TriMaster", niveau 2, 5 vies, score 80

    ┌──────────────────────────────────────────────────────────┐
    | GÉNÉRATION                                               |
    |                                                          |
    | getLevelConfig(2) → D1, config [A], 8 questions          |
    | getRandomTrio(distance: 1) → T-requin                    |
    |   { e: Requin, c: Teinte Rouge, r: Requin Rouge }        |
    | config aléatoire parmi [A] → A                           |
    | visible = [Requin, Teinte Rouge]                          |
    | masquée = Requin Rouge                                    |
    | distracteurs = 9 Réceptrices D1 (même distance)          |
    └──────────────────────────────────────────────────────────┘

    ┌──────────────────────────────────────────────────────────┐
    | AFFICHAGE                                                |
    |                                                          |
    | ┌─────────┐  (+)  ┌─────────┐  (=)  ┌─────────┐         |
    | | Requin |        |T. Rouge|        |  ???   |         |
    | └─────────┘        └─────────┘        └─────────┘         |
    |                                                          |
    | Choix : [Aigle Mir., Requin Rge, Lion Mir.,              |
    |          Panda Gris, Éléph. Rot., Renard Mir.,           |
    |          Lion Rge, Girafe Rot., Baleine Gris,            |
    |          Requin Mir.]                                     |
    |                                                          |
    | Timer : 30s                                               |
    └──────────────────────────────────────────────────────────┘

    ┌──────────────────────────────────────────────────────────┐
    | RÉPONSE (après 8 secondes)                               |
    |                                                          |
    | Le joueur tape "Requin Rge" → correctCardId match !      |
    |                                                          |
    | Calcul du score :                                         |
    |   basePoints           = 10                               |
    |   timeBonus(8, 30)     = 1.25  (8/30 = 27% < 50%)       |
    |   distanceMultiplier(1)= 1.0                              |
    |   score_question = 10 * 1.25 * 1.0 = 12.5 → 13 pts      |
    |                                                          |
    | Nouvel état :                                             |
    |   score: 80 + 13 = 93                                    |
```

```
    │   streak: 1                                          │
    │   correctAnswers: +1                                 │
    │   vies: 5 (inchangées)                               │
    └──────────────────────────────────────────────────────┘
```

Scénario 2 — Niveau 8, Config B, Mauvaise réponse

```
    Joueur : "TriMaster", niveau 8, 4 vies, score 520

    ┌──────────────────────────────────────────────────────┐
    │ GÉNÉRATION                                           │
    │                                                      │
    │ getLevelConfig(8) → D2, config [A,B], 10 questions   │
    │ getRandomTrio(distance: 2) → T-aigle-D2              │
    │   { e: R1-aigle-rot (Réceptrice D1 jouant rôle E),   │
    │     c: Teinte Bleu,                                  │
    │     r: R2-aigle-rot-bleu }                           │
    │ config aléatoire parmi [A,B] → B                     │
    │ visible = [R1-aigle-rot, R2-aigle-rot-bleu]          │
    │ masquée = Teinte Bleu                                │
    │ distracteurs = 4 câbles "couleur" + 5 autres câbles  │
    └──────────────────────────────────────────────────────┘

    ┌──────────────────────────────────────────────────────┐
    │ AFFICHAGE                                            │
    │                                                      │
    │ ┌─────────┐  (+)  ┌─────────┐  (=)  ┌─────────┐     │
    │ │ Aigle   │       │  ???    │       │ Aigle   │     │
    │ │ Rot.    │       │         │       │Rot.Bleu│     │
    │ └─────────┘       └─────────┘       └─────────┘     │
    │                                                      │
    │ Choix : 10 câbles mélangés                           │
    │ Timer : 40s                                          │
    └──────────────────────────────────────────────────────┘

    ┌──────────────────────────────────────────────────────┐
    │ RÉPONSE (après 15 secondes)                          │
    │                                                      │
    │ Le joueur tape "Niveaux de Gris"                     │
    │ → correctCardId = Teinte Bleu ≠ Niveaux de Gris      │
    │ → MAUVAISE RÉPONSE                                   │
    │                                                      │
    │ Animations :                                         │
    │   • "Niveaux de Gris" → bordure rouge + icône ✗      │
    │   • "Teinte Bleu" → bordure verte + icône ✓          │
    │     (révélation de la bonne réponse)                 │
    │                                                      │
    │ Nouvel état :                                        │
    │   score: 520 (inchangé, pas de points pour erreur)   │
    │   streak: 0 (remis à zéro)                           │
    │   wrongAnswers: +1                                   │
```

```
│   vies: 4 → 4 (livesPerWrong = 2, donc il faut       │
│              2 erreurs pour perdre 1 vie)             │
│                                                       │
│ Attente 1.8 secondes → question suivante              │
└───────────────────────────────────────────────────────┘
```

Scénario 3 — Niveau 16, Config C, Timeout

```
Joueur : "TriMaster", niveau 16, 2 vies, score 2100

┌───────────────────────────────────────────────────────┐
│ GÉNÉRATION                                            │
│                                                       │
│ getLevelConfig(16) → D3, config [B,C], 12 questions   │
│ getRandomTrio(distance: 3) → T-éléphant-D3            │
│   { e: R2-éléph-mir-rouge (Réceptrice D2 → rôle E),   │
│     c: Fragment 3,                                    │
│     r: R3-éléph-mir-rouge-frag }                      │
│ config aléatoire parmi [B,C] → C                      │
│ visible = [Fragment 3, R3-éléph-mir-rouge-frag]       │
│ masquée = R2-éléph-mir-rouge (une Réceptrice D2       │
│           qui joue le rôle d'Émettrice !)             │
│ distracteurs = 9 Réceptrices D2 (même distance)       │
└───────────────────────────────────────────────────────┘

Note sur les distracteurs :
  La carte masquée est R2-éléphant (card_type = "receptrice",
  distance_level = 2). Même si elle JOUE LE RÔLE d'Émettrice
  dans le trio D3, les distracteurs sont choisis par son TYPE
  SQL réel : "receptrice" + distance 2.

  → 9 autres Réceptrices D2 : Lion Mir. Rouge,
    Aigle Rot. Bleu, Requin Gris, etc.

┌───────────────────────────────────────────────────────┐
│ AFFICHAGE                                             │
│                                                       │
│ ┌─────────┐  (+)  ┌─────────┐  (=)  ┌─────────────┐   │
│ │   ???   │       │Fragmen. │       │  Éléphant   │   │
│ │         │       │   3     │       │Mir.Rge.Frag │   │
│ └─────────┘       └─────────┘       └─────────────┘   │
│                                                       │
│ Choix : 10 images (Réceptrices D2 mélangées)          │
│ Timer : 50s                                           │
│                                                       │
│ Le joueur hésite... les Réceptrices D2 se ressemblent │
│ beaucoup (toutes ont 2 transformations appliquées).   │
│                                                       │
│ 50... 40... 30... 20... 10... 5... 0 !                │
└───────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────┐
│ TIMEOUT                                                  │
│                                                          │
│ Le chrono atteint 0. Aucune image sélectionnée.          │
│                                                          │
│ Traitement identique à une mauvaise réponse PLUS :       │
│   • malus timeout : -5 points sur la session             │
│   • la bonne réponse est révélée (bordure verte)         │
│                                                          │
│ Nouvel état :                                            │
│   score: 2100 - 5 = 2095                                 │
│   streak: 0                                              │
│   wrongAnswers: +1                                       │
│   vies: 2 → 1 (livesPerWrong = 2 au niveau 16,           │
│               si c'est la 2ème erreur → -1 vie)          │
│                                                          │
│ DANGER : il ne reste qu'1 vie !                          │
└──────────────────────────────────────────────────────────┘
```

Scénario 4 — Fin de niveau réussie

```
Joueur : "TriMaster", niveau 7, après la 10ème question

┌──────────────────────────────────────────────────────────┐
│ BILAN DU NIVEAU                                          │
│                                                          │
│ Questions        : 10                                    │
│ Bonnes réponses  : 8                                     │
│ Mauvaises        : 2                                     │
│ Seuil requis     : 7/10                                  │
│ Résultat         : 8 >= 7 → RÉUSSI ✓                     │
│                                                          │
│ Score session    : 185 pts                               │
│ Meilleure série  : 5 bonnes consécutives                 │
│ Durée            : 4 min 32 sec                          │
│                                                          │
│ Étoiles (basées sur la précision) :                      │
│   >= 90% (9/10) → ☆☆☆                                    │
│   >= 70% (7/10) → ☆☆                                     │
│   < 70%         → ☆                                      │
│   8/10 = 80%    → ☆☆ (2 étoiles)                         │
│                                                          │
│ Sauvegarde :                                             │
│   → game_sessions : nouvelle entrée (score, correct,     │
│     wrong, passed=true, duration, maxStreak=5)           │
│   → user_level_progress : stars=2, bestScore=185,        │
│     completed=true                                       │
│   → user_profiles : current_level = 8,                   │
│     total_score += 185                                   │
│   → user_unlocked_cards : débloquer les cartes           │
│     rencontrées pendant le niveau                        │
```

```
    │                                                          │
    │ Navigation → TGameResultPage                              │
    └──────────────────────────────────────────────────────────┘
```

Scénario 5 — Fin de niveau échouée (plus de vies)

```
    Joueur : "TriMaster", niveau 12, question 6/10

    ┌──────────────────────────────────────────────────────────┐
    │ ÉTAT ACTUEL                                               │
    │                                                           │
    │ vies: 1                                                    │
    │ wrongAnswers: 3                                            │
    │ livesPerWrong: 2 (au niveau 12)                            │
    │                                                           │
    │ Le joueur donne une mauvaise réponse (4ème erreur).        │
    │ 4 erreurs / livesPerWrong(2) = 2 vies perdues au           │
    │ total. Il en avait 3 au début → 3 - 2 = 1.                 │
    │                                                           │
    │ MAIS il avait déjà 1 vie (pas 3) car il avait perdu        │
    │ des vies avant cette session. Donc :                       │
    │ lives = 1 - 1 = 0                                          │
    │                                                           │
    │ vies = 0 → GAME OVER pour cette session                    │
    └──────────────────────────────────────────────────────────┘

    ┌──────────────────────────────────────────────────────────┐
    │ FIN ANTICIPÉE                                             │
    │                                                           │
    │ Le niveau s'arrête IMMÉDIATEMENT (pas besoin               │
    │ d'attendre les 10 questions).                              │
    │                                                           │
    │ correctAnswers: 4                                          │
    │ threshold: 7                                               │
    │ 4 < 7 → ÉCHOUÉ ✗                                          │
    │                                                           │
    │ Sauvegarde :                                               │
    │   → game_sessions : passed=false, completed=false          │
    │   → user_profiles : current_level INCHANGÉ (pas +1)        │
    │   → user_profiles : total_score INCHANGÉ                   │
    │     (pas de points pour un niveau échoué)                  │
    │                                                           │
    │ Navigation → TGameResultPage (mode échec)                  │
    │   → Bouton "Réessayer" (même niveau)                       │
    │   → MAIS le joueur a 0 vies !                              │
    │   → Message : "Plus de vies ! Recharge dans X min."        │
    │   → Recharge automatique : +1 vie toutes les 30 min        │
    └──────────────────────────────────────────────────────────┘
```

---

12. Cas limites et règles spéciales

12.1. Pas assez de distracteurs

```
Situation : la base ne contient que 5 Réceptrices D3.
On en a besoin de 9.

┌──────────────────────────────────────────────────────────┐
│ Requête : 9 Réceptrices D3 sauf la bonne réponse         │
│ Résultat : seulement 4 cartes retournées                 │
│                                                          │
│ Le code accepte ce qu'il a :                             │
│   choices = [bonne réponse, 4 distracteurs]              │
│           = 5 choix au lieu de 10                        │
│                                                          │
│ L'interface s'adapte : la ScrollView affiche 5 images    │
│ au lieu de 10. Le jeu reste jouable.                     │
│                                                          │
│ Ce cas ne devrait arriver que pendant le développement   │
│ (base partiellement remplie).                            │
└──────────────────────────────────────────────────────────┘
```

12.2. Tous les trios déjà utilisés

```
Situation : session de 10 questions au niveau 3 (D1).
La base ne contient que 8 trios D1.
Questions 1 à 8 : chaque trio utilisé une fois.
Question 9 : excludeIds contient les 8 trios.

┌──────────────────────────────────────────────────────────┐
│ getRandomTrio(distance: 1, excludeIds: [8 IDs])          │
│ Requête retourne 0 résultat → data.isEmpty = true        │
│                                                          │
│ → Exception : "Aucun trio disponible pour distance 1"    │
│                                                          │
│ Solution actuelle : le GameSessionProvider attrape       │
│ l'erreur et passe en état "error".                       │
│                                                          │
│ Solution idéale : si on manque de trios, on ré-utilise   │
│ un trio déjà posé (vider les excludeIds les plus         │
│ anciens). Le joueur ne s'en rendra pas forcément         │
│ compte si le trio réapparaît avec une config             │
│ différente (A au lieu de B par exemple).                 │
└──────────────────────────────────────────────────────────┘
```

12.3. R1 comme Émettrice en D2 — qui est "Émettrice" ?

```
Confusion possible : dans un trio D2, l'emettrice_id
pointe vers R1, qui est de card_type = "receptrice".

┌──────────────────────────────────────────────────────────┐
│ Question : "R1 est-elle une Émettrice ou une             │
│            Réceptrice ?"                                 │
│                                                          │
│ Réponse : R1 est TOUJOURS de type "receptrice"           │
│           dans la table cards.                           │
│           Son card_type ne change JAMAIS.                │
│                                                          │
│ Mais dans le CONTEXTE du trio D2, elle JOUE LE RÔLE      │
│ d'Émettrice. C'est le champ emettrice_id du trio         │
│ qui définit le rôle, PAS le card_type de la carte.       │
│                                                          │
│ card_type    = identité permanente de la carte           │
│ emettrice_id = rôle temporaire dans un trio              │
│                                                          │
│ Le code s'en fiche : il lit le trio, charge les 3        │
│ cartes par leurs IDs, et les affiche selon la config.    │
│ Il ne regarde JAMAIS le card_type pour décider           │
│ qui est visible/masqué.                                  │
└──────────────────────────────────────────────────────────┘
```

12.4. Sélection de distracteurs pour une R qui joue E en config C

```
Situation : Trio D3, config C (trouver l'Émettrice).
L'émettrice du trio est R2 (card_type = "receptrice", distance = 2).

┌──────────────────────────────────────────────────────────┐
│ maskedCard = R2 (la carte à trouver)                     │
│ maskedCard.cardType = CardType.receptrice                │
│ maskedCard.distanceLevel = 2                             │
│                                                          │
│ getDistractors(correctCard: R2) :                        │
│   SELECT FROM cards                                      │
│   WHERE card_type = 'receptrice'    ← type réel de R2    │
│     AND distance_level = 2          ← même distance      │
│     AND id != R2-uuid                                    │
│   LIMIT 9                                                │
│                                                          │
│ Les 9 distracteurs sont d'autres Réceptrices D2.         │
│ C'est cohérent : le joueur cherche parmi des images      │
│ qui se RESSEMBLENT (toutes ont 2 transformations).       │
│                                                          │
│ Le joueur voit : C3 (Fragment) + R3 (Éléph. complet)     │
│ Il doit deviner : R2 (Éléph. avant fragmentation)        │
│ Les distracteurs : 9 autres images D2 similaires         │
└──────────────────────────────────────────────────────────┘
```

12.5. Câble sans catégorie (cas défensif)

```
Situation : un câble en base n'a pas de cable_category (null).

┌──────────────────────────────────────────────────────────┐
│ Le code vérifie :                                        │
│   if (correctCard.isCable                                │
│       && correctCard.cableCategory != null)              │
│                                                          │
│ Si cableCategory est null :                              │
│   → on SAUTE la stratégie en 2 temps                     │
│   → on tombe dans le cas général                         │
│   → SELECT par type + distance + exclusion               │
│   → les 9 distracteurs sont 9 câbles quelconques         │
│                                                          │
│ Ce cas ne devrait pas arriver en production              │
│ (chaque câble DOIT avoir une catégorie).                 │
│ Mais le code ne plante pas si ça arrive.                 │
└──────────────────────────────────────────────────────────┘
```

Résumé visuel final

```
┌──────────────────────────────────────────────────────────────────┐
│                        FLUX COMPLET                              │
│                                                                  │
│  Joueur appuie "Jouer"                                           │
│        │                                                         │
│        ▼                                                         │
│  getLevelConfig(niveau)                                          │
│        │                                                         │
│        ▼                                                         │
│  getRandomTrio(distance, exclure déjà posés)                     │
│        │                                                         │
│        ▼                                                         │
│  Future.wait(charger E, C, R en parallèle)                       │
│        │                                                         │
│        ▼                                                         │
│  Choisir config (A, B ou C) au hasard                            │
│        │                                                         │
│        ├── A : visible=[E,C] masquée=R                           │
│        ├── B : visible=[E,R] masquée=C                           │
│        └── C : visible=[C,R] masquée=E                           │
│        │                                                         │
│        ▼                                                         │
│  getDistractors(masquée) → 9 cartes même type                    │
│        │                                                         │
│        ├── Si Câble : 4 même catégorie + 5 autres                │
│        └── Sinon    : 9 même type + même distance                │
│        │                                                         │
```

```
│        ▼                                                    │
│  [masquée + 9 distracteurs].shuffle()                       │
│        │                                                    │
│        ▼                                                    │
│  Afficher : 2 visibles + "???" + 10 choix + timer           │
│        │                                                    │
│        ▼                                                    │
│  Joueur tape une image                                      │
│        │                                                    │
│        ├── Correct  → +score, +streak, question suivante    │
│        ├── Incorrect→ +erreurs, streak=0, révéler réponse   │
│        └── Timeout  → comme incorrect + malus -5 pts        │
│        │                                                    │
│        ▼                                                    │
│  Dernière question OU vies = 0 ?                            │
│        │                                                    │
│        ├── NON → question suivante (retour au début)        │
│        └── OUI → calcul résultat final                      │
│              │                                              │
│              ├── correct >= threshold → RÉUSSI              │
│              │     → +1 niveau, sauver score, étoiles       │
│              └── correct < threshold  → ÉCHOUÉ              │
│                    → même niveau, pas de score              │
└──────────────────────────────────────────────────────────────┘
```